Технический отчёт: ветка feat/synchronization

Репозиторий DmitriyZhariy/USV-Obstacle-Detection

---

1. Общее описание ветки

Назначение и решаемая задача

Ветка feat/synchronization решает задачу **постобработочной темпоральной синхронизации** видеопотоков, полученных с гетерогенного набора камер в ходе натурного эксперимента на борту БНСС (USV). Система записи включала три устройства: смартфон (центральная камера, phone_center) и два автомобильных регистратора (cam_left, cam_right). Поскольку все три устройства стартовали независимо и не имели общего источника синхросигнала (без PPS/GPS-дисциплинирования, без NTP-выравнивания часов), реальные временны́е смещения между потоками могут достигать десятков секунд. Цель ветки — точно вычислить эти смещения и сформировать единый синхронизированный манифест (synced_manifest.csv), который станет входными данными для последующей сегментации и детекции препятствий.

Что именно синхронизируется

Синхронизируются **видеофайлы** трёх камер: .mp4 (смартфон) и .avi (регистраторы). Механизм синхронизации — аудиальный: одновременно с запуском съёмки производился короткий звуковой хлопок (clap/clapboard), зафиксированный всеми тремя микрофонами. По совпадению этого острого звукового события в аудиодорожках вычисляется временно́е смещение (offset) между

потоками. IMU, GPS и другие сенсорные потоки в данной ветке **не синхронизируются** — это задача будущих этапов.

Отличия от main и feat/segmentation-dataset

Характеристика	main	feat/synchronization	feat/segmentation-dataset
Модули src/usv/	Отсутствуют (только __init__.py)	data_loader, preprocessing, visualization	Другой набор модулей
Ключевые зависимости	numpy, torch, opencv	+ ffmpeg-python, librosa, noisereduce	+ ultralytics, tqdm, opencv-contrib
Назначение	Базовая структура	Временна́я синхронизация видеопотоков	Работа с датасетом для сегментации/детекции
Статус configs/config.yaml	Пустой	Пустой	Пустой

main является «чистой» базовой веткой без реализованных модулей. feat/segmentation-dataset использует ultralytics (YOLO) и opencv-contrib-python — это ветка работы с обучающими данными. feat/synchronization полностью изолирована в решаемой задаче и не пересекается по коду с feat/segmentation-dataset.

---

2. Архитектура проекта

Структура директорий и файлов

```
USV-Obstacle-Detection/  (feat/synchronization)
├── configs/
│   └── config.yaml        # Пустой файл-заглушка (Hydra не настроен)
```

```
├── src/
│   └── usv/
│       ├── __init__.py
│       ├── data_loader/
│       │   └── manifest_generator.py   # Парсинг видеофайлов → CSV-мани
│       ├── preprocessing/
│       │   ├── audio_sync.py          # Ядро: AudioAligner (кросс-корреляция
│       │   ├── sync_pipeline.py       # Простой пайплайн (первые файлы)
│       │   ├── resync_full_dataset.py  # Продвинутый пайплайн (весь датас
│       │   └── test_denoise_sync.py    # Тестовый скрипт денойзинга
│       └── visualization/
│           ├── check_sync.py          # Matplotlib: волновые формы
│           ├── verify_sync.py         # OpenCV: мультистримовый плеер
│           ├── verify_peaks.py        # Консенсусный оффсет по пикам
│           └── analyze_claps.py       # Детальный диагностический инструм
├── reports/                          # Генерируемые CSV и графики
├── models/                           # Каталог моделей (пустой)
├── pyproject.toml
└── main.py                           # Заглушка точки входа
```

Структура следует модифицированному шаблону Cookiecutter Data Science. Реальные видеоданные (data/) не хранятся в репозитории.

Компоненты и их ответственность

VideoManifestGenerator (data_loader/manifest_generator.py) — сканирует дерево каталогов data/raw, находит все видеофайлы (.mp4, .avi, .mov), определяет принадлежность камере по имени каталога или шаблону имени файла, извлекает точную длительность через ffprobe, парсит временну́ю метку из имени файла телефона (формат VID_YYYYMMDD_HHMMSS). Результат — pandas.DataFrame, сохраняемый в reports/data_manifest.csv.

AudioAligner (preprocessing/audio_sync.py) — ядро системы синхронизации. Загружает аудиодорожку из видеофайла через librosa, применяет цепочку очистки сигнала (шумоподавление + полосовая фильтрация) и вычисляет временно́е смещение методом взаимной кросс-корреляции (FFT).

SyncPipeline (preprocessing/sync_pipeline.py) — простой пайплайн первого поколения. Синхронизирует **первые** видеофайлы регистраторов относительно первого файла телефона, затем последовательно «состыковывает» остальные файлы каждого потока.

FullDatasetResync (preprocessing/resync_full_dataset.py) — расширенный пайплайн второго поколения. Синхронизирует **каждый** видеофайл регистратора индивидуально против непрерывного таймлайна телефона, используя «умный поиск» нужного фрагмента телефонной записи. Поддерживает скользящее окно поиска и порог корреляции для отбраковки ненадёжных совпадений.

Модули visualization/ — инструменты ручной верификации: check_sync.py строит графики волновых форм (Matplotlib), verify_sync.py запускает синхронный просмотр трёх потоков (OpenCV), verify_peaks.py реализует консенсусный алгоритм определения оффсета по множеству пиков, analyze_claps.py выполняет глубокий диагностический анализ (огибающая Гильберта, onset-функция).

Механизм синхронизации на уровне кода

Синхронизация реализована как **однопоточный автономный (offline) пайплайн**. Никаких потоков (threading), очередей (queue), pub/sub, ROS-топиков или asyncio не используется. Все операции выполняются последовательно над файловой системой. Механизм передачи данных между этапами — CSV-файлы (pandas.DataFrame).

Поток данных

```
data/raw/
 ├── center_phone/ (VID_YYYYMMDD_HHMMSS.mp4)
 ├── left_cam/     (MOVI0000.avi, MOVI0001.avi, ...)
 └── right_cam/    (MOVI0000.avi, MOVI0001.avi, ...)
       |
       ▼  VideoManifestGenerator.create_manifest()
reports/data_manifest.csv   ← время старта/конца только для phone (из и
```

```
        │
        ▼  AudioAligner.load_audio() + clean_signal() + find_offset()
        │  (librosa → noisereduce → scipy Butterworth → scipy fftconvolve)
        │
        ▼  SyncPipeline.run()  /  FullDatasetResync.run()
reports/synced_manifest.csv   ← calc_start_time / calc_end_time для ВСЕХ ка
        │
        ▼  (следующий этап — детекция/сегментация)
```

---

3. Используемые зависимости

Полный список (из `pyproject.toml`)

Библиотека	Версия	Роль в синхронизации
`ffmpeg-python`	≥ 0.2.0	Обёртка над `ffprobe` для извлечения длительности и метаданных видео
librosa	≥ 0.11.0	Загрузка аудиодорожки из видеофайла, ресемплирование до 16 кГц, onset-анализ
`noisereduce`	≥ 3.0.3	Спектральное шумоподавление (Spectral Gating) для устранения шума двигателя/ветра
scipy	≥ 1.16.3	butter + lfilter (полосовой фильтр), fftconvolve (кросс-корреляция), find_peaks
`numpy`	≥ 2.3.5	Операции над массивами сигналов, Z-нормализация, статистика
pandas	≥ 2.3.3	Хранение и обработка манифестов, работа с временны́ми метками (pd.Timestamp)
`opencv-python`	≥ 4.11.0	Покадровое воспроизведение синхронизированных потоков для визуальной верификации
matplotlib	≥ 3.10.7	Построение графиков волновых форм, огибающих, onset-функций
`hydra-core`	≥ 1.3.2	Управление конфигурацией (интеграция не реализована, `config.yaml` пустой)
torch, torchvision	≥ 2.9.1 / 0.24.1	Зарезервированы для последующих модулей детекции (не используются в синхронизации)

Зависимости, добавленные специально для синхронизации

По сравнению с main, ветка feat/synchronization добавляет три библиотеки:

- **ffmpeg-python** — критична для фазы генерации манифеста: без неё невозможно получить точную длительность видеофайлов без заголовков с корректными временны́ми метками.
- **librosa** — ключевая для AudioAligner: обеспечивает единообразную загрузку аудио из .mp4 и .avi с ресемплированием.
- **noisereduce** — добавлена для борьбы со специфическим шумом USV (шум двигателя, гидродинамические вибрации), маскирующим полезный сигнал хлопка.

---

4. Конкретные реализации и объяснение кода

Шаг 1 — Генерация манифеста (VideoManifestGenerator)

Класс сканирует директорию data/raw рекурсивно, определяет тип камеры по имени каталога или файла (phone / left / right), получает длительность через ffmpeg.probe:

```
probe = ffmpeg.probe(str(file_path))
video_stream = next(s for s in probe['streams'] if s['codec_type'] == 'video')
duration = float(video_stream['duration'])
```

Критическая особенность смартфона: стандарт Android сохраняет в имени файла **время завершения** записи. Поэтому время начала вычисляется как:

```
end_time = file_ts          # из VID_YYYYMMDD_HHMMSS
start_time = end_time - timedelta(seconds=duration)
```

Для регистраторов (cam_left, cam_right) временны́е метки **не извлекаются** — поля calc_start_time и calc_end_time остаются None до этапа синхронизации.

Шаг 2 — Очистка аудиосигнала (AudioAligner.clean_signal)

Цепочка обработки:

```
# 1. Spectral Gating (noisereduce): убирает постоянный фоновый шум (мо
data_clean = nr.reduce_noise(y=data, sr=self.sr, prop_decrease=0.7, n_fft=512)

# 2. Butterworth bandpass 2500–7500 Hz (5-й порядок)
# ВАЖНО: была 300–4000 Hz → поднята до 2500–7500 для изоляции резко
b, a = scipy.signal.butter(5, [2500/8000, 7500/8000], btype='band')
data_filtered = scipy.signal.lfilter(b, a, data_clean)
```

Подъём нижней границы с 300 Гц до 2500 Гц — ключевое решение, зафиксированное в коде как CRITICAL FIX: на USV структурные вибрации и удары волн создают мощные низкочастотные «тумпы» (thuds), которые на прежнем диапазоне маскировали пик хлопка и сдвигали корреляционный максимум.

Шаг 3 — Вычисление временно́го смещения (AudioAligner.find_offset)

Метод реализует стандартную нормализованную взаимную кросс-корреляцию через FFT:

```
# Z-score нормализация (устойчивость к разным уровням громкости)
ref_norm = (ref_sig - np.mean(ref_sig)) / (np.std(ref_sig) + 1e-9)
tgt_norm = (tgt_sig - np.mean(tgt_sig)) / (np.std(tgt_sig) + 1e-9)

# FFT-свёртка: эффективнее O(N log N) против O(N²) прямой корреляции
correlation = scipy.signal.fftconvolve(ref_norm, tgt_norm[::-1], mode='full')

lags = np.arange(-len(tgt_norm) + 1, len(ref_norm))
```

```
lag_samples = lags[np.argmax(correlation)]
time_offset = lag_samples / self.sr   # лаг в сэмплах → секунды
```

Математически:

$$\hat{\tau} = \arg\max_{\tau} \sum_{n} x_{\text{ref}}[n] \cdot x_{\text{tgt}}[n - \tau]$$

, где $\hat{\tau}$ — искомое смещение в отсчётах. Положительное значение означает, что целевой сигнал (регистратор) **отстаёт** от эталонного (телефона).

Шаг 4 — Простой пайплайн (SyncPipeline)

Первое поколение: синхронизирует **только первый файл** каждого регистратора, затем стыкует последующие последовательно:

```
audio_offset, corr = self.aligner.find_offset(ref_audio, tgt_audio, apply_cleani
total_offset = audio_offset + self.HARDWARE_LATENCY  # HARDWARE_LATE
first_cam_start_time = master_start_time + timedelta(seconds=total_offset)

# Последовательная стыковка
for idx, row in cam_videos.iterrows():
    df.at[idx, 'calc_start_time'] = current_start_time
    df.at[idx, 'calc_end_time'] = current_start_time + timedelta(seconds=row['du
    current_start_time += timedelta(seconds=row['duration_sec'])
```

Ограничение: накопление ошибки — если первый файл синхронизирован с погрешностью, ошибка переносится на все последующие.

Шаг 5 — Расширенный пайплайн (FullDatasetResync)

Второе поколение устраняет ограничение через индивидуальную синхронизацию каждого файла:

```
# Умный поиск нужного фрагмента телефона
def get_master_audio_segment(self, phone_df, target_global_time, duratio
    candidates = phone_df[phone_df['calc_end_time'] > target_global_time]
```

```
    ref_row = candidates.sort_values('calc_start_time').iloc
    offset_in_file = (target_global_time - ref_row['calc_start_time']).total_second
    y, _ = librosa.load(path, sr=16000, offset=max(0, offset_in_file),
                        duration=duration_needed, mono=True)
    return y, ref_row['filename']
```

Для каждого файла регистратора: если файл не первый — окно поиска ±25 с вокруг ожидаемого начала; если первый — широкое окно 120 с. После нахождения оффсета проверяется порог: `if corr > 10.0` — иначе fallback к последовательной стыковке.

Обработка рассинхронизаций, задержек и потерь данных

- **Потери (gap в телефонных записях)**: функция `get_master_audio_segment` возвращает `None`, если следующий файл телефона начинается более чем на 20 с позже запрашиваемого времени → fallback «пристыковать к предыдущему».
- **Низкое качество корреляции**: порог `corr > 10.0` отфильтровывает ненадёжные совпадения → тот же fallback.
- **Верификация**: инструмент `verify_peaks.py` использует консенсусный алгоритм — находит все значимые пики в обоих сигналах, строит матрицу разностей, кластеризует по округлённым значениям до 0.1 с и выбирает моду как консенсусный оффсет.
- **Аппаратная задержка**: изначально предполагалась как отдельная константа `HARDWARE_LATENCY`, но в коммите `080da58` она была обнулена (`= 0.0`) после вывода, что кросс-корреляция уже учитывает её.

Интеграция с модулем детекции

На выходе пайплайна формируется файл `reports/synced_manifest.csv` / `reports/synced_manifest_full.csv`, содержащий столбцы `camera_id`, `path`, `calc_start_time`, `calc_end_time` для каждого видеофайла. Модуль детекции/сегментации может использовать этот манифест для:

- Побитового извлечения синхронных кадров из трёх потоков по любому абсолютному времени (метод `VisualVerifier.find_video_covering_time`);
- Открытия видеофайла с нужным смещением (`cv2.CAP_PROP_POS_MSEC`) для покадровой обработки.

На уровне архитектуры связь односторонняя и асинхронная: `feat/synchronization` производит манифест, `feat/segmentation-dataset` потребляет его.

Конфигурационные параметры

Параметр	Значение	Где задан	Влияние
sample_rate	16000 Гц	AudioAligner.__init__	Разрешение смещения: 1/16000 = 62.5 мкс. Снижение → меньше точность
bandpass_low	2500 Гц	clean_signal()	Нижняя граница изоляции хлопка; снижение → больше вибрационных шумов
bandpass_high	7500 Гц	clean_signal()	Верхняя граница; превышение Nyquist (8000 Гц при 16кГц) защищено проверкой
prop_decrease	0.7	clean_signal() → noisereduce	Интенсивность шумоподавления (70%); значение 0.9 в тесте слишком агрессивно
SEARCH_WINDOW_SEC	25 с	FullDatasetResync.__init__	Ширина окна поиска; при больших сдвигах между файлами нужно увеличить
corr_threshold	10.0	resync_full_dataset.py	Жёстко закодирован; слишком мал или велик → пропуск файлов или ложные совпадения
HARDWARE_LATENCY	0.0 с	SyncPipeline.__init__	Аппаратная задержка; обнулена, т.к. поглощается кросс-корреляцией
Загружаемая длительность	40 с	SyncPipeline.run()	Гарантирует захват обоих хлопков (начало + конец); слишком мало → промах

---

5. Анализ развития ветки

История коммитов

Дата	SHA	Сообщение	Что добавлено
2025-11-27	343a9d6	chore: initial commit	Базовая структура проекта
2025-12-01	1f865bd	build: migrate to uv and restructure	Переход на uv, пакетная структура src/usv/
2025-12-01	0478a05	feat: implement AudioAligner	Первая версия AudioAligner со знаком смещения
2025-12-01	8a433cf	feat: finalize manifest generator	Подтверждена логика таймстемпа телефона (конец = имя файла)
2025-12-01	02c387a	fix: enable bandpass filter (300–3000 Hz)	Первый полосовой фильтр для улучшения корреляции
2025-12-02	c6edd27	fix: invert audio offset calculation logic	Исправлен знак (direction) смещения
2025-12-02	108fb75	feat: finalize sync pipeline	Гибридная синхронизация: корреляция + HARDWARE_LATENCY
2025-12-02	080da58	fix: remove manual latency correction	HARDWARE_LATENCY = 0.0 (убрана ручная поправка)
2025-12-05	4123e0e	feat: start scaling sync func on all videos	FullDatasetResync, подъём полосы до 2500–7500 Гц

Ветка активно разрабатывалась в течение примерно 8 дней (27 ноября — 5 декабря 2025 г.). Эволюция хорошо прослеживается: от базового прототипа до полноценного двухпоколенного пайплайна с несколькими итерациями исправления знаков и настройки фильтров.

Текущий статус

Ветка находится в состоянии **рабочего прототипа с незавершённой масштабируемостью**. Основные алгоритмы реализованы и отлажены; SyncPipeline (простой пайплайн) можно считать функционально завершённым. FullDatasetResync запущен в разработку, но последний коммит (feat: start scaling) указывает, что тестирование на полном датасете только началось. Инфраструктурные элементы (configs/config.yaml, интеграция Hydra) не реализованы.

Проблемы и ограничения

Функциональные:

- test_denoise_sync.py вызывает aligner.apply_bandpass_filter(), которого нет в актуальном AudioAligner — это сломанный скрипт, несовместимый с текущим API.
- Порог корреляции corr > 10.0 в FullDatasetResync жёстко закодирован без документации по калибровке; при другом типе звукового маркера или записи может быть некорректным.
- verify_sync.py содержит хардкод if leftname != 'MOVI0025.avi': continue — инструмент заточен под конкретный файл датасета.
- configs/config.yaml пустой, hydra-core в зависимостях есть, но не используется.

Архитектурные:

- Накопление ошибки в SyncPipeline: последовательная стыковка после первого смещения накапливает дрейф при длинных сессиях.
- Нет оценки неопределённости: пайплайн не сохраняет значение корреляции (corr) и ширину пика в манифесте —

невозможно постфактум определить качество синхронизации каждого файла.
- Нет автоматического тестирования синхронизации: pytest настроен, но тестов нет.

Ограничения для real-time USV:

- Система полностью **офлайновая**: предназначена для постобработки записей, не для работы в реальном времени.
- Метод хлопка требует ручной процедуры перед каждой записью и не масштабируется на автономные миссии.
- При записи в ветреную или штормовую погоду шум может превышать уровень хлопка — алгоритм не имеет механизма отказа кроме порога корреляции.

Перспективы интеграции в main

Конфликты: Ветка main не содержит модулей src/usv/, поэтому слияние не создаст конфликтов в коде. Единственное пересечение — pyproject.toml, который потребует ручного объединения зависимостей.

Необходимые доработки перед мержем:

1. Заполнить configs/config.yaml параметрами синхронизации и подключить Hydra в пайплайны.
2. Исправить test_denoise_sync.py: заменить вызов несуществующего apply_bandpass_filter на clean_signal.
3. Добавить сохранение поля corr_value в выходной манифест для оценки качества.
4. Написать unit-тесты для AudioAligner.find_offset на синтетических сигналах с известным смещением.
5. Убрать хардкоды файлов (MOVI0025.avi) из verify_sync.py.
6. Документировать калибровку порога corr > 10.0.

После решения этих задач ветка может быть смержена в main как этап preprocessing в общей пайплайн-архитектуре: raw data → синхронизация (эта ветка) → сегментация/детекция (feat/segmentation-dataset) → локализация.